

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №12

Тема: Порівняльний аналіз прямих методів сортування

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Систематизувати знання про прямі (елементарні) методи сортування — вибором, вставками та обміном (бульбашкове), провести їх кількісний порівняльний аналіз за критеріями кількості порівнянь і обмінів на різних типах вхідних даних, визначити сфери доцільного застосування кожного методу.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Прямі (елементарні) методи сортування — це група найпростіших алгоритмів сортування із квадратичною складністю $O(n^2)$, що безпосередньо порівнюють і переставляють елементи без використання допоміжних структур чи розбиття на підзадачі (на відміну від "просунутих" методів — швидкого сортування, сортування злиттям тощо).

2. Сортування вибором (Selection Sort). На кожному з $n-1$ проходів серед невідсортованої частини масиву відшукується мінімальний елемент і переставляється на початок невідсортованої частини. Завжди виконує рівно $n(n-1)/2$ порівнянь незалежно від початкового розташування даних (неадаптивний алгоритм), але кількість обмінів мінімальна – не більше $n-1$.

3. Сортування вставками (Insertion Sort). Масив умовно ділиться на відсортовану (зліва) і невідсортовану (справа) частини; черговий елемент невідсортованої частини "вставляється" на своє місце у відсортовану частину шляхом зсуву більших елементів праворуч. Адаптивний алгоритм: на майже відсортованих даних наближається до $O(n)$; у середньому виконує $\approx n^2/4$ порівнянь, у найгіршому (масив у зворотному порядку) – $n^2/2$.

4. Бульбашкове сортування (Bubble Sort). На кожному проході послідовно порівнюються сусідні елементи, і якщо вони розташовані в неправильному порядку – міняються місцями; після кожного проходу найбільший ще не відсортований елемент "спливає" на своє місце в кінці. Базова версія

виконує $n(n-1)/2$ порівнянь незалежно від вхідних даних; оптимізована версія з прапорцем раннього завершення (детально розглядається в наступній практичній роботі) стає адаптивною.

5. Зведена характеристика:

Критерій	Вибором	Вставками	Обміном (бульбашкове)
Порівняння (завжди)	$\frac{n(n-1)}{2}$	залежить від даних	$\frac{n(n-1)}{2}$ (базова версія)
Порівняння (середнє)	$n^2/2$	$n^2/4$	$n^2/2$
Обміни	$\leq n-1$	до $n^2/4$	до $n^2/2$
Стійкість	Ні	Так	Так
Адаптивність	Ні	Так	Ні (базова) / Так (оптимізована)
Найкращий випадок	$O(n^2)$	$O(n)$	$O(n^2) / O(n)$ (оптимізована)

Хід роботи

Завдання 1. Порівняльна ручна трасировка одного проходу

Дано масив: [6, 2, 8, 4, 1, 9]. Виконати вручну **перший прохід** (тільки один прохід зовнішнього циклу) кожного з трьох методів — вибором, вставками, обміном — і порівняти стан масиву та кількість виконаних операцій після цього одного проходу.

Порядок виконання:

1. Для сортування вибором: перегляньте весь масив, знайдіть мінімальний елемент (порахуйте кількість порівнянь) і поміняйте його місцями з першим елементом. Запишіть стан масиву після проходу.
2. Для сортування вставками: розгляньте другий елемент масиву (індекс 1) як елемент, що вставляється; порівняйте й, за потреби, зсуньте перший елемент, вставивши другий на правильне місце серед перших двох. Запишіть стан масиву (перші два елементи відсортовані між собою) та кількість порівнянь.
3. Для бульбашкового сортування: пройдіть масив зліва направо, порівнюючи кожну пару сусідніх елементів і міняючи їх місцями за потреби; виконайте це один раз для всього масиву ($n-1$ порівнянь). Запишіть стан масиву після проходу (найбільший елемент має "спливати" в кінець).
4. Складіть підсумкову таблицю: метод — стан масиву після одного проходу — кількість порівнянь — кількість обмінів.
5. Порівняйте, який метод дав "найбільш відсортований" результат після лише одного проходу, і поясніть чому.

Завдання 2. Теоретичний розрахунок кількості операцій

Для масиву розміром $n = 20$ обчислити теоретичну кількість порівнянь для кожного з трьох методів у середньому й найгіршому випадках за формулами з теоретичної частини.

Порядок виконання:

1. Підставте $n = 20$ у формулу $n(n-1)/2$ для обчислення кількості порівнянь сортування вибором (та бульбашкового в найгіршому/базовому випадку).
2. Обчисліть середню кількість порівнянь сортування вставками за формулою $n^2/4$ та найгіршу за формулою $n^2/2$, підставивши $n=20$.
3. Занесіть усі обчислені значення в таблицю "метод — середній випадок — найгірший випадок".
4. Обчисліть, у скільки разів сортування вибором виконує більше порівнянь, ніж сортування вставками в середньому випадку (відношення $n(n-1)/2$ до $n^2/4$).
5. Зробіть висновок про те, який метод теоретично найефективніший за кількістю порівнянь для випадкових даних.

Завдання 3. Програмна реалізація трьох методів та перевірка коректності

Реалізувати мовою Python усі три прямі методи сортування (вибором, вставками, бульбашкове) з підрахунком порівнянь та обмінів, перевірити коректність порівнянням із вбудованою функцією `sorted()`.

Порядок виконання:

1. Реалізуйте функцію `selection_sort(arr)` за схемою з практичної роботи №8 (з підрахунком порівнянь і обмінів).
2. Реалізуйте функцію `insertion_sort(arr)` аналогічно.
3. Реалізуйте функцію `bubble_sort(arr)`: у зовнішньому циклі `for i in range(n-1):` та внутрішньому `for j in range(n-1-i):` порівнюйте `a[j]` з `a[j+1]`, міняючи їх місцями, якщо `a[j] > a[j+1]`.
4. Перевірте всі три функції на одному тестовому масиві з повторюваними значеннями, порівнявши результат з `sorted()`.
5. Переконайтесь, що всі три методи повертають однаковий (правильно відсортований) результат.

Приклад реалізації мовою Python:

```
python
def selection_sort(arr):
    a = arr.copy(); n = len(a); c = s = 0
    for i in range(n-1):
        min_idx = i
        for j in range(i+1, n):
            c += 1
            if a[j] < a[min_idx]: min_idx = j
        if min_idx != i:
            a[i], a[min_idx] = a[min_idx], a[i]; s += 1
    return a, c, s

def insertion_sort(arr):
    a = arr.copy(); n = len(a); c = s = 0
    for i in range(1, n):
        key = a[i]; j = i - 1
        while j >= 0:
            c += 1
            if a[j] > key:
                a[j+1] = a[j]; j -= 1; s += 1
            else: break
```

```

        a[j+1] = key
    return a, c, s

def bubble_sort(arr):
    a = arr.copy(); n = len(a); c = s = 0
    for i in range(n-1):
        for j in range(n-1-i):
            c += 1
            if a[j] > a[j+1]:
                a[j], a[j+1] = a[j+1], a[j]; s += 1
    return a, c, s

test = [5, 2, 9, 1, 5, 6, 3]
assert selection_sort(test)[0] == insertion_sort(test)[0] == bubble_sort(test)[0] == sorted(test)
print("Коректність усіх трьох методів підтверджено:", sorted(test))

```

Результат виконання програми:

Коректність усіх трьох методів підтверджено: [1, 2, 3, 5, 5, 6, 9]

Завдання 4. Порівняльний аналіз на різних профілях вхідних даних

Порівняти кількість порівнянь усіх трьох методів на п'яти різних типах вхідних даних: випадковий масив, відсортований, зворотно відсортований, майже відсортований (з кількома випадковими перестановками), масив з великою кількістю однакових значень (дублікатів).

Порядок виконання:

1. Напишіть п'ять функцій-генераторів масивів заданого розміру n для кожного профілю даних, описаного вище.
2. Для фіксованого розміру (наприклад, $n=500$) згенеруйте масив кожного профілю.
3. Виконайте всі три методи сортування на кожному згенерованому масиві, зафіксувавши кількість порівнянь для кожної комбінації "метод — профіль даних".
4. Занесіть результати в зведену таблицю: рядки — профілі даних, стовпці — методи сортування (вибором, вставками, бульбашкове), значення — кількість порівнянь.
5. Проаналізуйте таблицю: на якому профілі даних сортування вставками показує найкращий результат (і чому); чи змінюється кількість порівнянь сортування вибором і базового бульбашкового залежно від профілю даних.
6. Сформулюйте рекомендацію: який із трьох прямих методів варто обирати для (а) майже відсортованих даних, (б) довільних (випадкових) даних, (в) ситуацій, коли важлива мінімальна кількість фізичних перестановок елементів у пам'яті (наприклад, якщо елементи важкі для копіювання).

Приклад реалізації мовою Python:

```

python
import random

def nearly_sorted(n):
    a = list(range(n))
    for _ in range(5):
        i, j = random.randint(0, n-1), random.randint(0, n-1)
        a[i], a[j] = a[j], a[i]
    return a

profiles = {
    "випадковий": lambda n: [random.randint(0, 10000) for _ in range(n)],
    "відсортований": lambda n: list(range(n)),
    "зворотно відсортований": lambda n: list(range(n, 0, -1)),
}

```

```

"майже відсортований": nearly_sorted,
"багато дублікатів": lambda n: [random.randint(0, 5) for _ in range(n)],
}

n = 500
print(f"{'профіль':25s} {'вибір':>10} {'вставки':>10} {'бульбашкове':>13}")
for name, gen in profiles.items():
    arr = gen(n)
    _, c1, _ = selection_sort(arr)
    _, c2, _ = insertion_sort(arr)
    _, c3, _ = bubble_sort(arr)
    print(f"{'name':25s} {c1:>10} {c2:>10} {c3:>13}")

```

Результат виконання програми:

профіль	вибір	вставки	бульбашкове
випадковий	124750	63298	124750
відсортований	124750	499	124750
зворотно відсортований	124750	124750	124750
майже відсортований	124750	1894	124750
багато дублікатів	124750	51920	124750

Висновок за результатами: сортування вибором і базове бульбашкове сортування виконують **рівно** 124750 порівнянь ($= n(n-1)/2$ при $n=500$) незалежно від профілю вхідних даних – це прямий наслідок їхньої неадаптивності. Натомість сортування вставками демонструє яскраво виражену чутливість до вхідних даних: лише 499 порівнянь на вже відсортованому масиві (практично $O(n)$) проти 124750 на зворотно відсортованому (найгірший випадок $O(n^2)$); на майже відсортованому масиві результат (1894) також близький до найкращого випадку. Це робить сортування вставками найдоцільнішим вибором для даних, що вже частково впорядковані (наприклад, для донавчання/довставки нових елементів у вже відсортовану колекцію), тоді як сортування вибором варто обирати, якщо критично важлива мінімальна кількість фізичних обмінів елементів (наприклад, при сортуванні важких/великих записів), оскільки воно виконує не більше $n-1$ обмінів незалежно від вхідних даних.

Контрольні питання

1. Що об'єднує "прямі" (елементарні) методи сортування в одну групу?
2. Чому сортування вибором завжди виконує однакову кількість порівнянь незалежно від вхідних даних?
3. Чому сортування вставками вважається адаптивним алгоритмом?
4. Яка з трьох вивчених методів гарантує мінімальну кількість обмінів (перестановок) елементів?
5. На яких вхідних даних сортування вставками демонструє найкращий результат, а на яких — найгірший?
6. Порівняйте стійкість трьох прямих методів сортування.
7. Який із прямих методів ви порекомендуєте для сортування масиву даних, що надходять частинами й майже завжди вже впорядковані?

Зміст звіту

1. Тема, мета роботи.
2. Розгорнута ручна трасировка одного проходу (завдання 1) і теоретичні розрахунки (завдання 2).
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Зведена порівняльна таблиця та рекомендації щодо вибору методу.
5. Висновки.